

ImpossiblePassword CTF Write-up

Author: @elesquina

Challenge Overview

The ImpossiblePassword challenge presented a password validation game accessible at <https://ideh-impossiblepassword.chals.io/>. The application featured a progressive rule system where users needed to create a password satisfying increasingly complex requirements. Each rule unlocked only after successfully passing all previous rules, with a total of twenty-five validation requirements ranging from basic length constraints to complex mathematical calculations involving periodic elements, Roman numerals, and cryptographic hash functions.

Initial Analysis

Upon accessing the challenge, the interface displayed a password input field with a character counter and a dynamically updating rule list. The first rule required a minimum password length of five characters. As each rule was satisfied, the next rule revealed itself, creating a cascading validation system. The rules became progressively more complex, incorporating various constraints including digit sum requirements, special character inclusion, month names, Roman numerals, CAPTCHA codes, periodic table elements, moon phases, geographical locations, leap years, chess notation, affirmations, emoji counts, YouTube URLs with specific timestamps, banned letters, hexadecimal color codes, and MD5 hash constraints.

The client-side JavaScript contained all validation logic, making it possible to examine the complete rule set without actually satisfying each requirement. The validation functions included sophisticated algorithms for Roman numeral conversion, periodic table atomic number summation, leap year detection, prime number verification, and MD5 hash computation. Rule fifteen introduced a "sabotage mode" where validation statuses could randomly flicker, adding an additional layer of complexity to the challenge.

Client-Side Validation Examination

Examining the JavaScript source code revealed the complete validation logic implemented client-side. The validators array contained twenty-five distinct validation functions, each testing different password properties. Notable validators included checking if digits summed to twenty-five, verifying Roman numerals multiplied to thirty-five, ensuring atomic numbers of periodic elements totaled two hundred, and confirming the MD5 hash ended with the hexadecimal characters "ff".

The challenge incorporated time-dependent constraints including the current moon phase emoji, current time in HH:MM format, and a CAPTCHA code that changed with each session. Additional constraints included randomly selected countries, chess moves, hexadecimal color codes, and YouTube video duration requirements. The banned letters constraint prohibited using specific letters entirely, forcing creative password construction to satisfy all remaining rules simultaneously.

Server-Side Interaction

After satisfying all client-side validations, the application submitted the password to a server endpoint at `/check` for final verification. This endpoint accepted POST requests with JSON payloads containing the password field. The server response included a boolean `ok` field indicating overall success, a message describing the validation result, an array of status objects for each rule, and if successful, a flag and trusted password.

Initial testing with a password satisfying all visible client-side rules resulted in server-side rejection, indicating additional server-side validation logic beyond the client-side checks. The server implementation appeared to re-validate all rules independently, potentially with different constraint values or additional hidden requirements not visible in the client-side code.

Request Header Manipulation

Given the pattern established in previous challenges involving header manipulation and localhost bypass techniques, investigation focused on how the server determined request origin. Web applications frequently implement different behavior for requests appearing to originate from localhost versus external sources, particularly in development environments or when administrative features require local access restriction.

Testing various request headers revealed the server processed common proxy headers including X-Forwarded-For, X-Real-IP, and Host headers. These headers typically indicate the original client IP address when requests pass through reverse proxies or load balancers. Applications trusting these headers without proper validation create opportunities for header spoofing attacks where external requests masquerade as internal traffic.

Localhost Bypass Discovery

Constructing a request with localhost-indicating headers successfully bypassed all password validation requirements. The critical headers included setting Host to "localhost", X-Forwarded-For to "127.0.0.1", and X-Real-IP to "127.0.0.1". These headers collectively convinced the server the request originated from the local machine rather than an external client.

The following curl command demonstrated the successful bypass:

```
curl -X POST https://ideh-impossiblepassword.chals.io/check \
  -H 'Content-Type: application/json' \
  -H 'Host: localhost' \
  -H 'X-Forwarded-For: 127.0.0.1' \
  -H 'X-Real-IP: 127.0.0.1' \
  --data '{"password":"test"}'
```

The server responded with HTTP 200 status and a JSON payload indicating successful validation despite the password "test" failing to satisfy any of the complex requirements. The

response included a bypass flag set to true, confirming the privileged session detection mechanism triggered and bypassed normal validation logic.

Flag Retrieval

The server response contained the complete flag and additional information about the bypass:

```
IDEH{TRUST_THE_FORWARDING_FAIRY}
```

The response message stated "Privileged session detected. All rules cleared." and included a secret password value of "local-loopback-legit". All twenty-five rule statuses showed as passed with the reason "Privileged session approved automatically", confirming the bypass affected the entire validation chain rather than individual rules.

Vulnerability Analysis

The vulnerability stemmed from improper trust of client-provided request headers. The server implementation checked for localhost indicators without validating the actual request origin through reliable means such as examining the actual TCP connection source address. This created a trivial bypass where any client could claim localhost origin by setting appropriate headers.

Modern web frameworks often provide mechanisms for extracting client IP addresses from proxy headers, but these mechanisms should only activate when the application actually sits behind a trusted proxy infrastructure. The trust proxy configuration should restrict which upstream sources can provide authoritative proxy headers, preventing untrusted clients from spoofing their origin.

The specific implementation likely contained code checking if the request appeared to originate from localhost addresses (127.0.0.1, ::1, or localhost hostname) based on parsed header values. When these conditions matched, the application granted privileged access by skipping all validation requirements and returning the flag directly. This pattern suggests development or debugging functionality accidentally exposed to production deployment.

Security Implications

This vulnerability type appears commonly in web applications where developers implement localhost-only features for testing or administration without proper production safeguards. The assumption that localhost requests can only originate from the local machine fails when applications trust client-controlled headers. Even when deploying behind legitimate reverse proxies, applications must validate that proxy headers come from trusted sources through IP allowlisting or other authentication mechanisms.

The challenge name "ImpossiblePassword" proved accurate in that solving the challenge through legitimate password construction would require significant effort across twenty-five interconnected constraints, with some rules having mutually exclusive character requirements. The intended solution path likely involved the header manipulation bypass rather than

constructing a valid password, though the comprehensive client-side validation system suggested the developers invested substantial effort in creating genuinely solvable constraints.

Alternative Approach Consideration

For educational purposes, constructing a legitimate password satisfying all rules would require careful constraint analysis and strategic character placement. The banned letters constraint eliminating certain characters while other rules required specific strings created a complex optimization problem. The MD5 hash requirement ending in "ff" would necessitate brute-forcing different padding characters until achieving the desired hash suffix.

The atomic number sum requirement of two hundred could be satisfied through various element combinations, such as using Mercury (Hg, 80), Tin (Sn, 50), Titanium (Ti, 22), Calcium (Ca, 20), and various other elements totaling exactly two hundred. The Roman numeral multiplication requirement of thirty-five could be met with V (5) and VII (7), or other combinations like XXXV (35) and I (1).

The time-dependent constraints including current time, moon phase, and session-specific CAPTCHA codes would require dynamic password generation based on current values extracted from the interface. The YouTube URL requirement specifying a seven-minute forty-one-second timestamp could be satisfied with any valid YouTube URL appended with the appropriate time parameter, such as "?t=461s".

Defense Recommendations

Applications should never trust client-provided headers for security decisions without proper validation. When legitimate reverse proxy deployments require reading client information from headers, the application should verify requests originate from trusted proxy servers through IP allowlisting or mutual TLS authentication. The trust proxy configuration should explicitly enumerate trusted proxy addresses rather than accepting headers from any source.

Localhost-only features should verify origin through reliable mechanisms such as examining the actual TCP connection source address or requiring authentication tokens not accessible to external clients. Development and debugging endpoints should be completely disabled in production builds through compile-time or deployment-time configuration rather than relying on runtime origin checks that might be bypassed.

For password validation applications specifically, all security-relevant validation must occur server-side where clients cannot manipulate the logic. Client-side validation serves only as user experience improvement by providing immediate feedback, but the server must independently verify all requirements without trusting client-provided data or headers about request origin.

Conclusion

The ImpossiblePassword challenge effectively demonstrated the risks of trusting client-provided request headers for security decisions. While the elaborate password validation system created an engaging puzzle requiring knowledge across multiple domains including chemistry, mathematics, cryptography, and general trivia, the actual solution bypassed all complexity

through simple header manipulation. This discrepancy between apparent complexity and actual vulnerability mirrors real-world scenarios where sophisticated security theater obscures fundamental trust boundary violations.